



Teste Unitário

Nome: Marilene da Silva Araujo;

Turma: Desenvolvimento de Sistemas;

Nome da Equipe: Inventory Masters;

1. Suíte de testes unitários - Projeto: TCC_Inventory_Masters_Kinect.Tests

Projeto Alvo: TCC_Inventory_Masters_Kinect

Camada Testada: Repositório de Dados (Repository)

Framework Utilizado: xUnit (framework de testes)

Objetivo da Suíte: Validar de forma automatizada os comportamentos definidos na interface para persistência, consulta, isolamento por empresa e resiliência a exceções do banco de dados SQLite relacionados às medições volumétricas e históricos de ocupação do Kinect.

2. Testes implementados

Abaixo estão listados os casos de teste desenvolvidos e implementados utilizando o framework **xUnit** e o padrão **AAA (Arrange, Act, Assert)** para os componentes:

Casos de Teste - KinectRepository

01, SalvarMedicao_MedicaoValida_SalvaComSucesso

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (configurar uma instância válida do repositório com a empresa de teste e um objeto de medição volumétrica válido contendo volume e data/hora).

- **Act**

- ↓
- Executar o comportamento que será testado (tentar salvar a medição volumétrica invocando o método correspondente SalvarMedicao no repositório).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (validar se a operação foi concluída com sucesso sem lançar nenhuma exceção).

02. SalvarMedicao_ComEmpresaConfigurada_AtribuiEmpresaESalva

- **Arrange**

- ↓

- *Preparar os dados e condições necessárias (instanciar o repositório com uma empresa específica e criar um objeto de medição volumétrica válido).*

- **Act**

- ↓

- *Executar o comportamento que será testado (salvar a medição invocando `SalvarMedicao` e em seguida consultar o registro salvo utilizando o método `ObterUltimasMedicoes`).*

- **Assert**

- ↓

- *Verificar se o resultado corresponde ao esperado (validar que a medição foi salva e que o campo da empresa gravado corresponde exatamente à empresa configurada no repositório).*

03. `SalvarMedicao_ErroNoBanco_CapturaExcecaoELoga`

- **Arrange**

- ↓

- *Preparar os dados e condições necessárias (configurar o repositório e preparar um objeto de medição nulo ou inválido para forçar uma falha de persistência interna).*

- **Act**

- ↓

- *Executar o comportamento que será testado (tentar adicionar a medição inválida invocando o método `SalvarMedicao` no repositório).*

- **Assert**

- ↓

- *Verificar se o resultado corresponde ao esperado (validar se a exceção gerada foi capturada e tratada pelo mecanismo de segurança interno, sem propagar falhas).*

04. `ObterUltimasMedicoes_QuantidadeValida_RetornaListaLimitada`

- **Arrange**

- ↓

- *Preparar os dados e condições necessárias (inicializar o repositório com a empresa de teste devidamente configurada).*

- **Act**

- ↓

- Executar o comportamento que será testado (invocar o método `ObterUltimasMedicoes` passando uma quantidade limite válida de registros).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (validar que a lista retornada não é nula e respeita rigorosamente o limite máximo de itens especificado).

05. `ObterUltimasMedicoes_ErroNoBanco_RetornaListaVazia`

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (configurar o repositório e passar parâmetros de consulta inválidos para forçar um cenário de falha ou exceção interna).

- **Act**

- ↓
- Executar o comportamento que será testado (invocar o método `ObterUltimasMedicoes` com quantidade negativa).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (validar que a exceção foi tratada de forma resiliente e que uma lista vazia do tipo correto foi retornada).

01. `ObterMedicoesEmOrdemCrescente_ParametrosValidos_RetornaOrdenadoCrescente`

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (inicializar o repositório com a empresa de teste para consulta de dados ordenados).

- **Act**

- ↓
- Executar o comportamento que será testado (invocar o método `ObterMedicoesEmOrdemCrescente` informando a quantidade, o usuário e a empresa).

- **Assert**

- ↓

-
- *Verificar se o resultado corresponde ao esperado (validar que a lista retornada está organizada de forma crescente com base nos identificadores/dados das medições).*

02. ObterMedicoesEmOrdemCrescente_ErroNoBanco_RetornaListaVazia

- **Arrange**

- ↓
- *Preparar os dados e condições necessárias (configurar o repositório e definir parâmetros nulos ou inválidos para simular um cenário de exceção de consulta).*

- **Act**

- ↓
- *Executar o comportamento que será testado (invocar o método ObterMedicoesEmOrdemCrescente com parâmetros nulos).*

- **Assert**

- ↓
- *Verificar se o resultado corresponde ao esperado (validar que a falha foi capturada com segurança e o método retornou uma lista vazia).*

08. SalvarHistorico_HistoricoValido_SalvaComSucesso

- **Arrange**

- ↓
- *Preparar os dados e condições necessárias (configurar o repositório com a empresa e instanciar um registro de histórico de ocupação válido contendo o ID da medição e status).*

- **Act**

- ↓
- *Executar o comportamento que será testado (tentar salvar o histórico invocando o método correspondente SalvarHistorico no repositório).*

- **Assert**

- ↓
- *Verificar se o resultado corresponde ao esperado (validar se a operação foi executada com sucesso sem lançar exceções).*

09. SalvarHistorico_ComEmpresaConfigurada_AtribuiEmpresaESalva

- **Arrange**

- ↓

-
- Preparar os dados e condições necessárias (instanciar o repositório com a empresa de teste e criar um objeto de histórico de ocupação válido).

- **Act**

- ↓
- Executar o comportamento que será testado (salvar o histórico invocando `SalvarHistorico` e em seguida buscar os últimos registros salvos via `ObterUltimosHistoricos`).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (validar que o histórico foi persistido corretamente e que o campo da empresa associada confere com a configuração).

10. `SalvarHistorico_ErroNoBanco_CapturaExcecaoELoga`

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (configurar o repositório e definir um objeto de histórico nulo ou inválido para forçar uma falha no contexto).

- **Act**

- ↓
- Executar o comportamento que será testado (tentar salvar o histórico inválido invocando o método `SalvarHistorico` no repositório).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (validar se a exceção gerada foi capturada e tratada internamente, evitando quebras na aplicação).

11. `ObterHistoricoPorEspaco_EspacoExistente_RetornaHistoricoFiltrado`

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (inicializar o repositório com a empresa de teste e definir um ID de espaço/medição existente).

- **Act**

- ↓

- Executar o comportamento que será testado (invocar o método *ObterHistoricoPorEspaco* passando o ID do espaço correspondente).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (validar que a lista retornada contém apenas os registros de histórico filtrados estritamente pelo ID do espaço especificado).

12. *ObterHistoricoPorEspaco_ErroNoBanco_RetornaListaVazia*

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (configurar o repositório e definir um identificador de espaço inválido para forçar uma exceção na busca).

- **Act**

- ↓
- Executar o comportamento que será testado (invocar o método *ObterHistoricoPorEspaco* com o parâmetro inválido).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (validar que a exceção foi tratada de forma segura e o retorno foi uma lista vazia).

13. *ObterUltimosHistoricos_QuantidadeValida_RetornaListaLimitada*

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (inicializar o repositório com a empresa de teste devidamente configurada).

- **Act**

- ↓
- Executar o comportamento que será testado (invocar o método *ObterUltimosHistoricos* informando uma quantidade limite de registros válida).

- **Assert**

- ↓



- Verificar se o resultado corresponde ao esperado (validar que a lista de históricos retornada não é nula e respeita o limite de quantidade especificado).

14. ObterUltimosHistoricos_ErroNoBanco_RetornaListaVazia

• Arrange

- ↓
- Preparar os dados e condições necessárias (configurar o repositório e passar uma quantidade negativa ou inválida para forçar um cenário de erro interno).

• Act

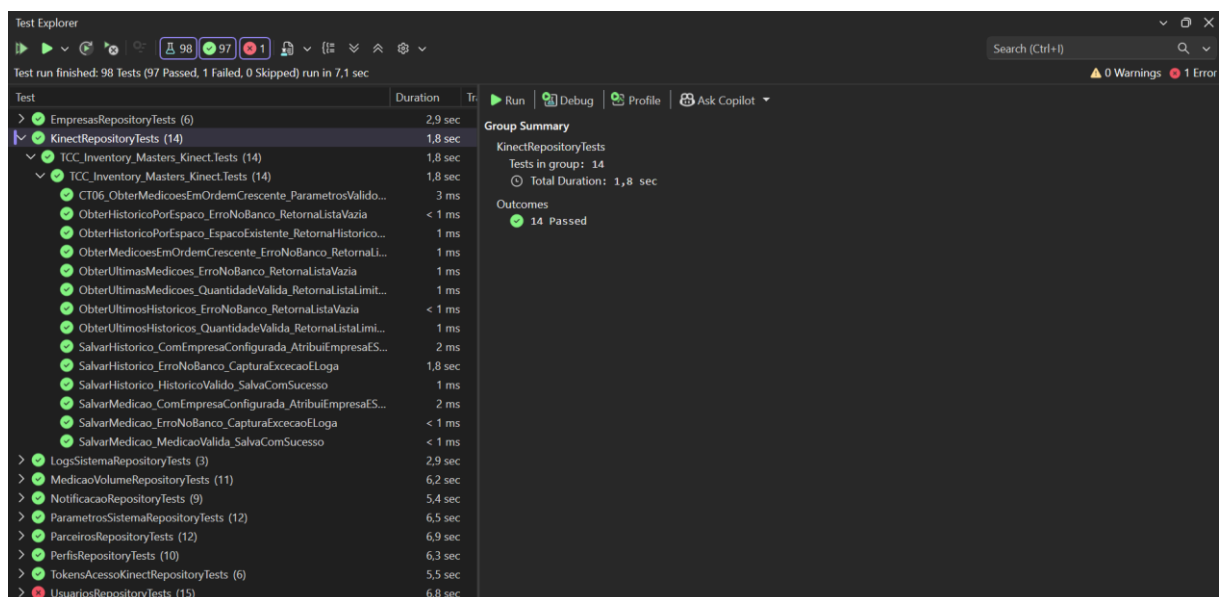
- ↓
- Executar o comportamento que será testado (invocar o método ObterUltimosHistoricos com o parâmetro inválido).

• Assert

- ↓
- Verificar se o resultado corresponde ao esperado (validar que a falha foi capturada com sucesso e que o método retornou uma lista vazia).

3. Evidências da execução

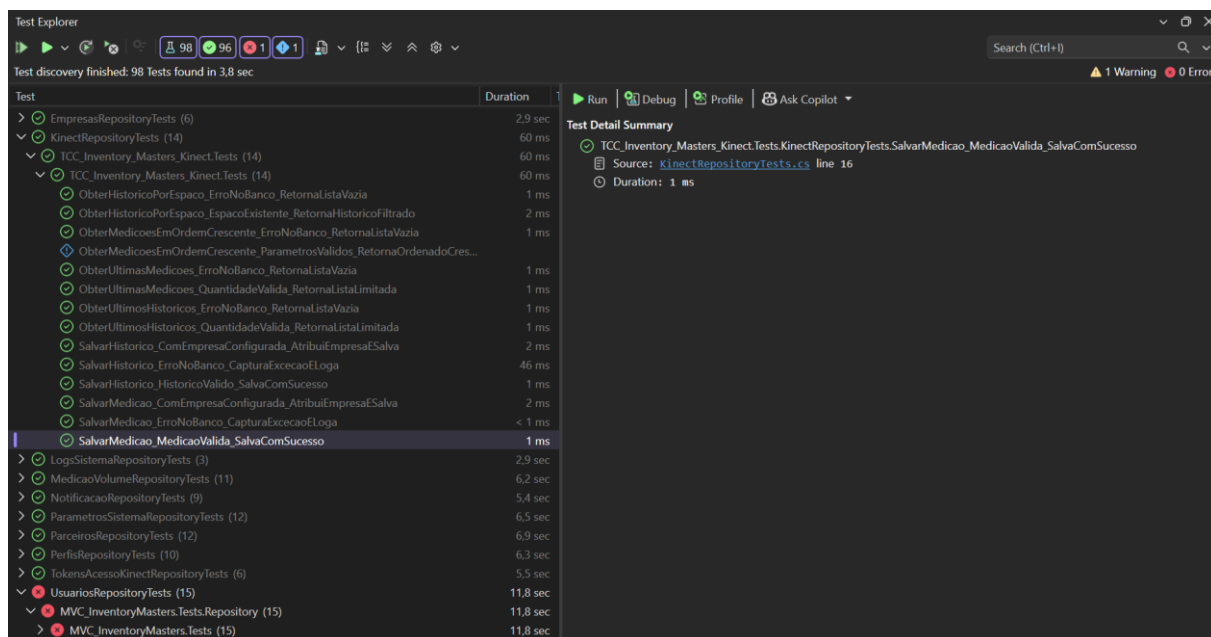
Componente: KinectRepositoryTests



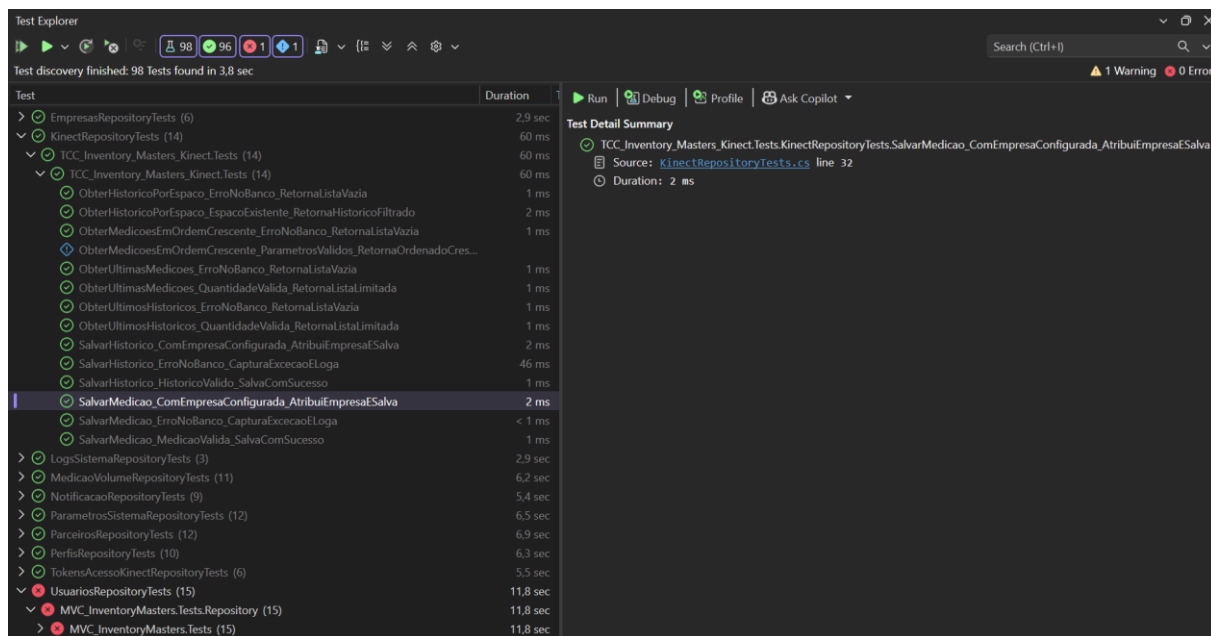
Casos de Teste - KinectRepository



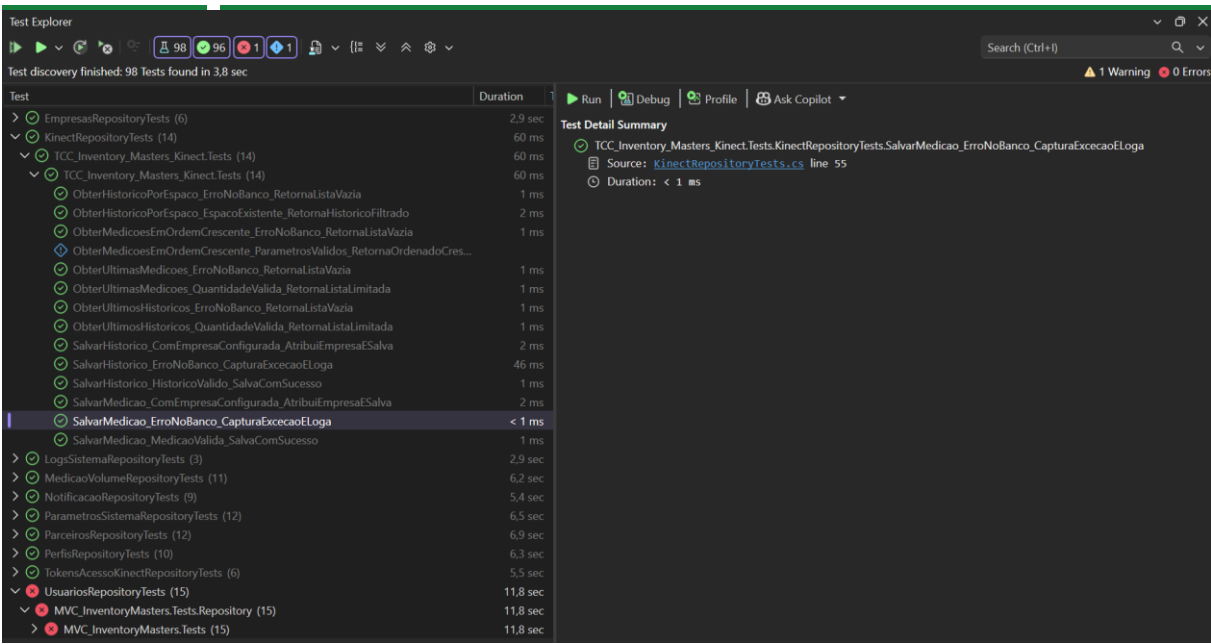
01. SalvarMedicao_MedicacaoValida_SalvaComSucesso



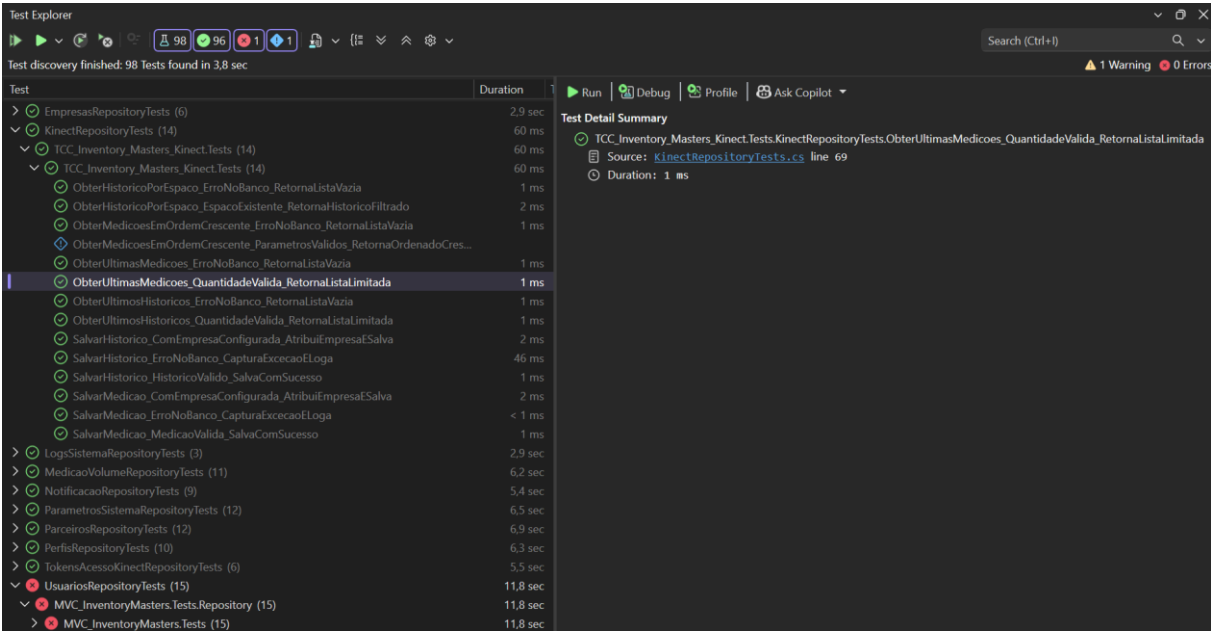
02. SalvarMedicao_ComEmpresaConfigurada_AtribuiEmpresaESalva



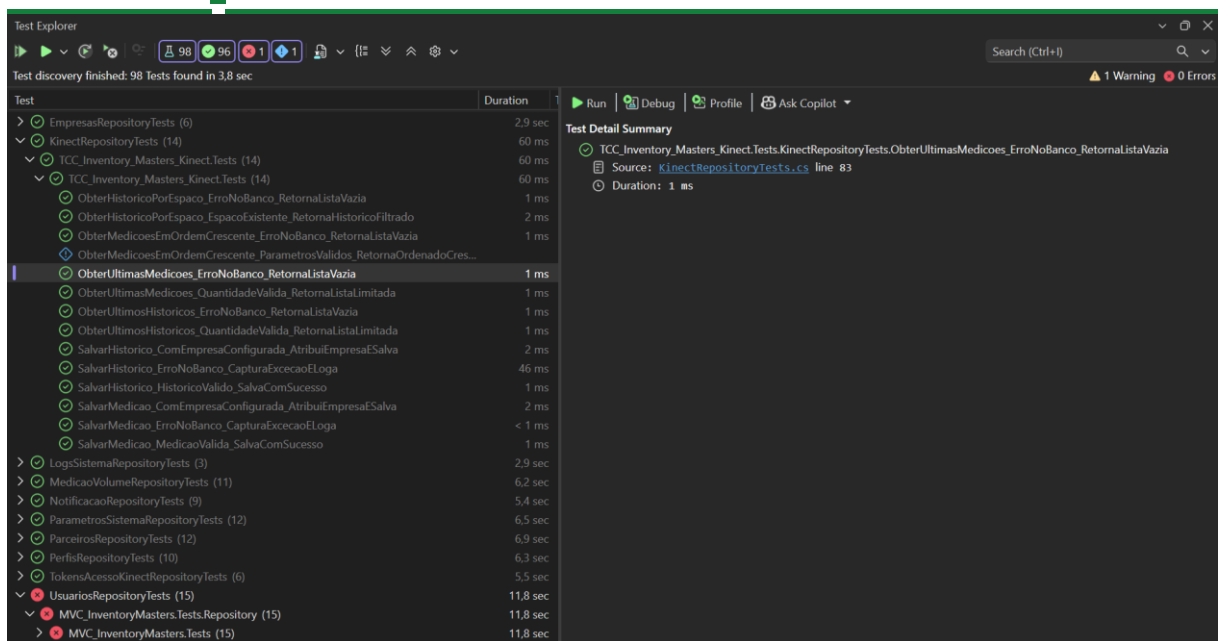
03. SalvarMedicao_ErroNoBanco_CapturaExcecaoELoga



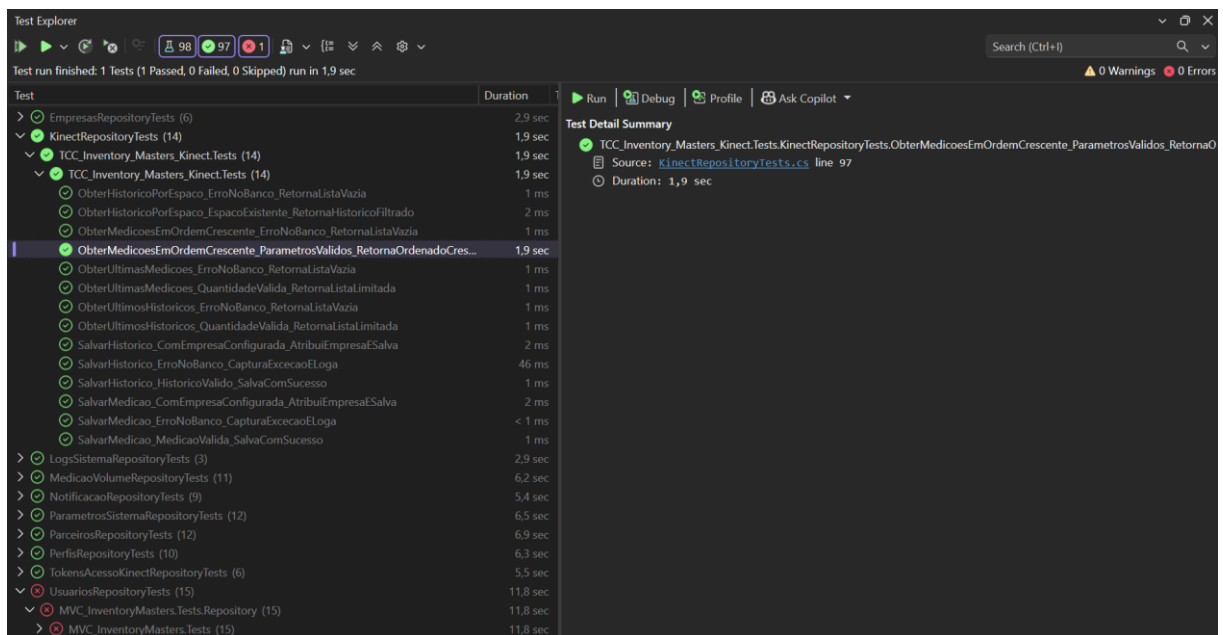
04. ObterUltimasMedicoes_QuantidadeValida_RetornaListaLimitada



05. ObterUltimasMedicoes_ErroNoBanco_RetornaListaVazia



06. ObterMedicoesEmOrdemCrescente_ParametrosValidos_RetornaOrdenadoCrescente

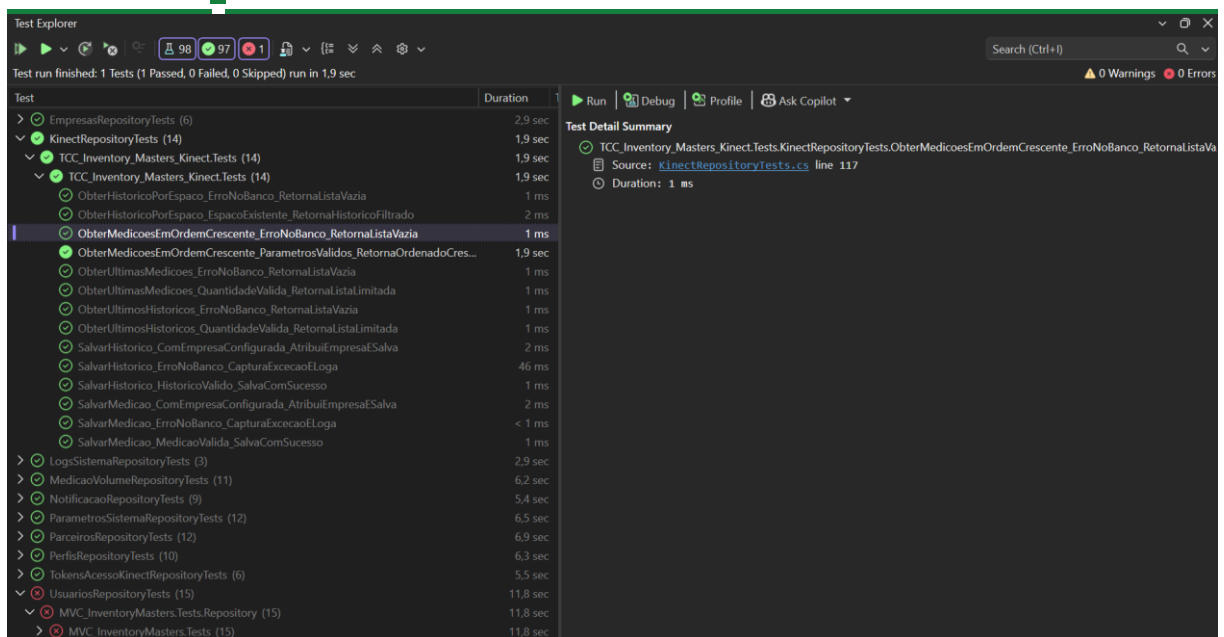


07. ObterMedicoesEmOrdemCrescente_ErroNoBanco_RetornaListaVazia

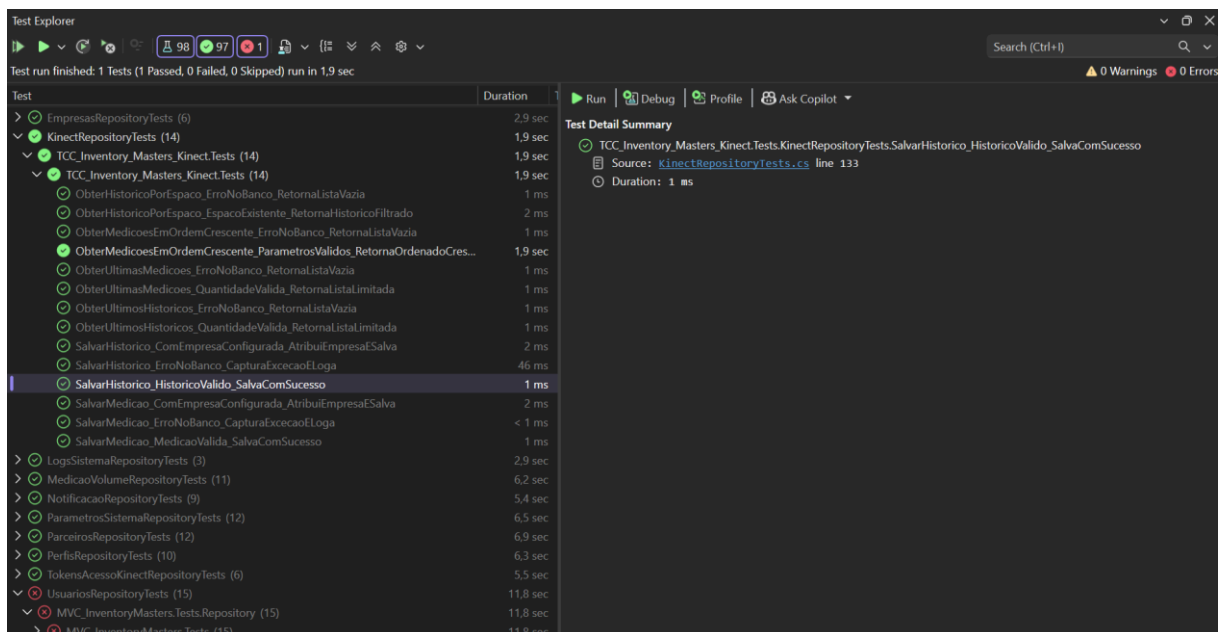


INVENTORY MASTERS

Gestão de Estoque Inteligente



08. SalvarHistorico_HistoricoValido_SalvaComSucesso

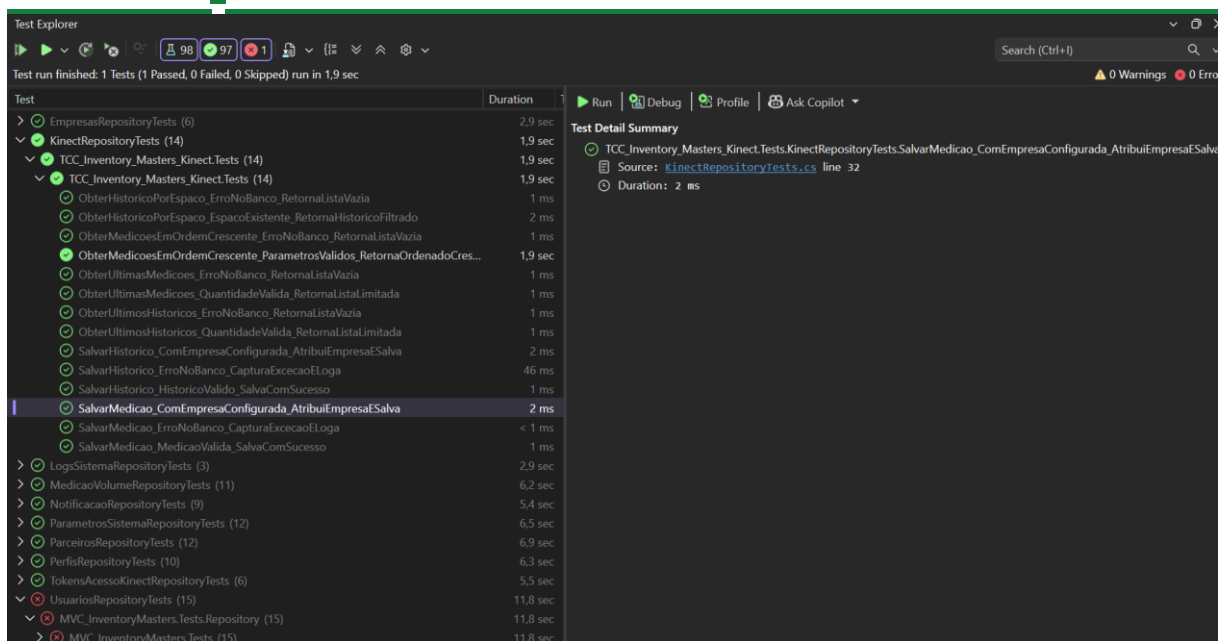


09. SalvarHistorico_ComEmpresaConfigurada_AtribuiEmpresaESalva

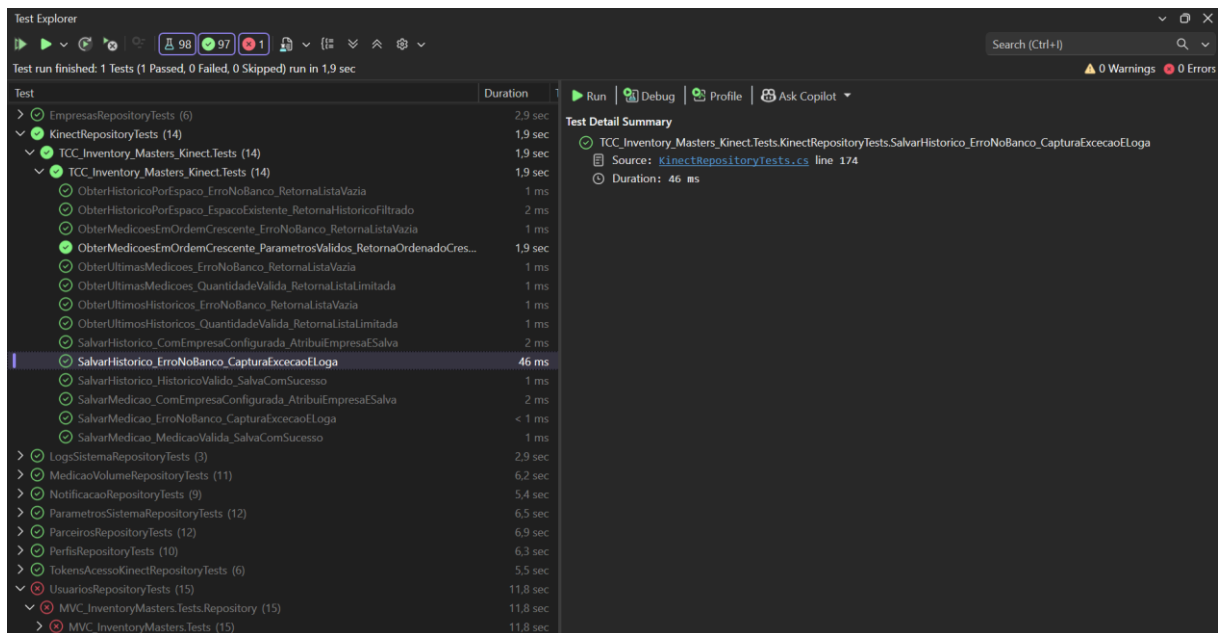


INVENTORY MASTERS

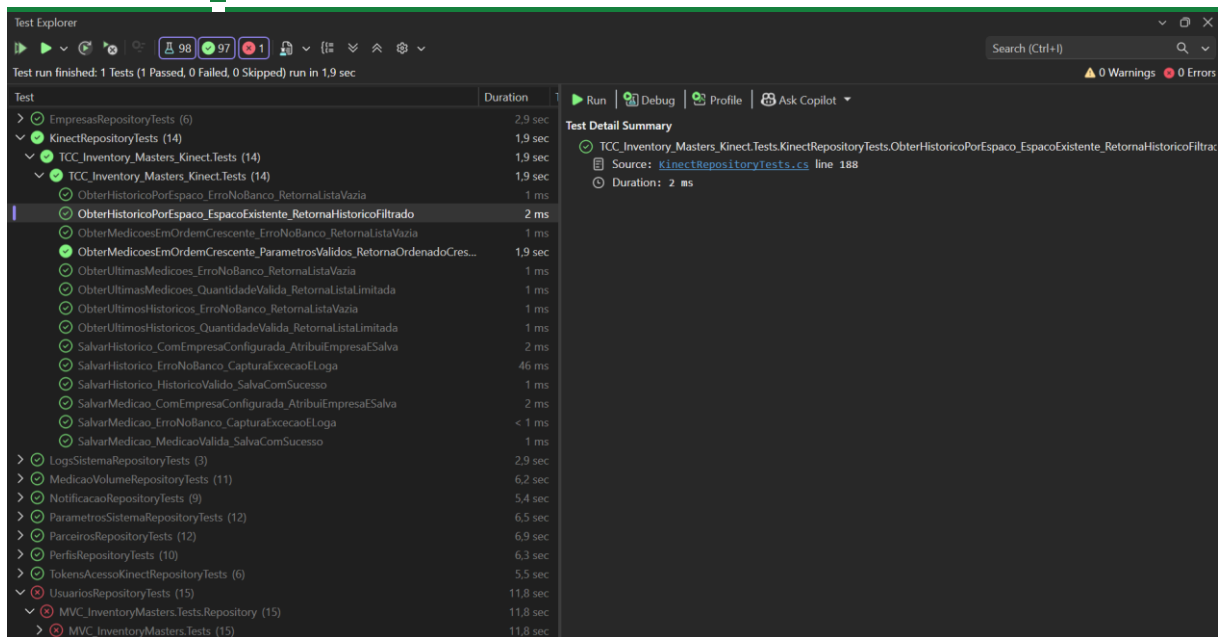
Gestão de Estoque Inteligente



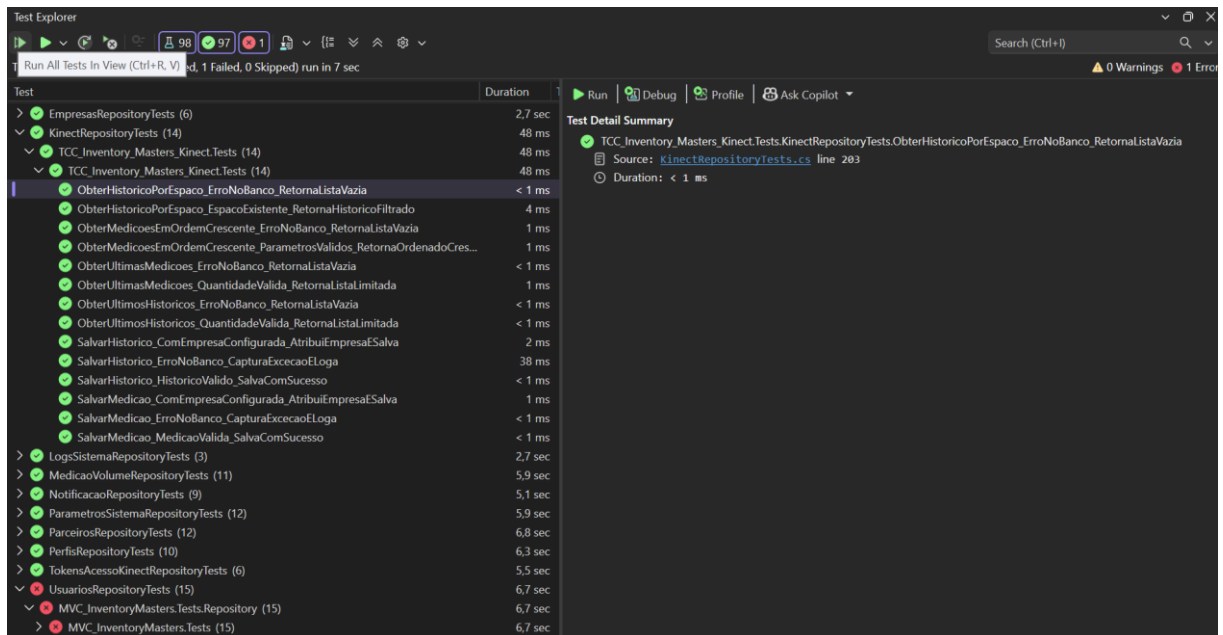
10. SalvarHistorico_ErroNoBanco_CapturaExcecaoELoga



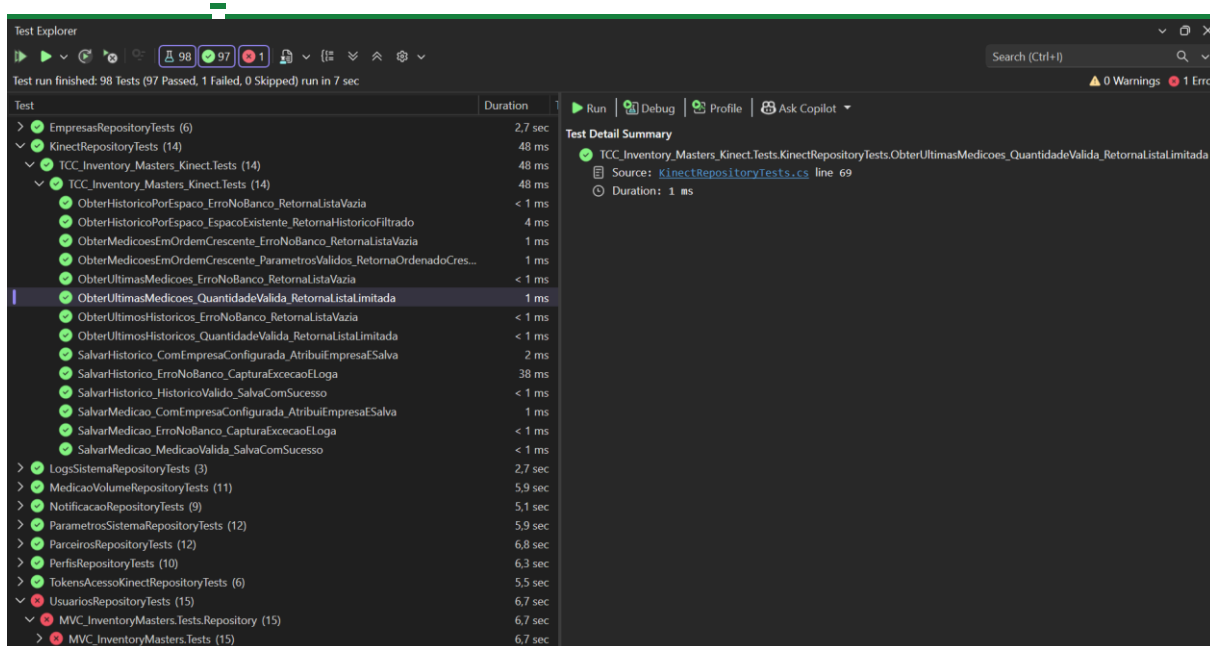
11. ObterHistoricoPorEspaco_EspacoExistente_RetornaHistoricoFiltrado



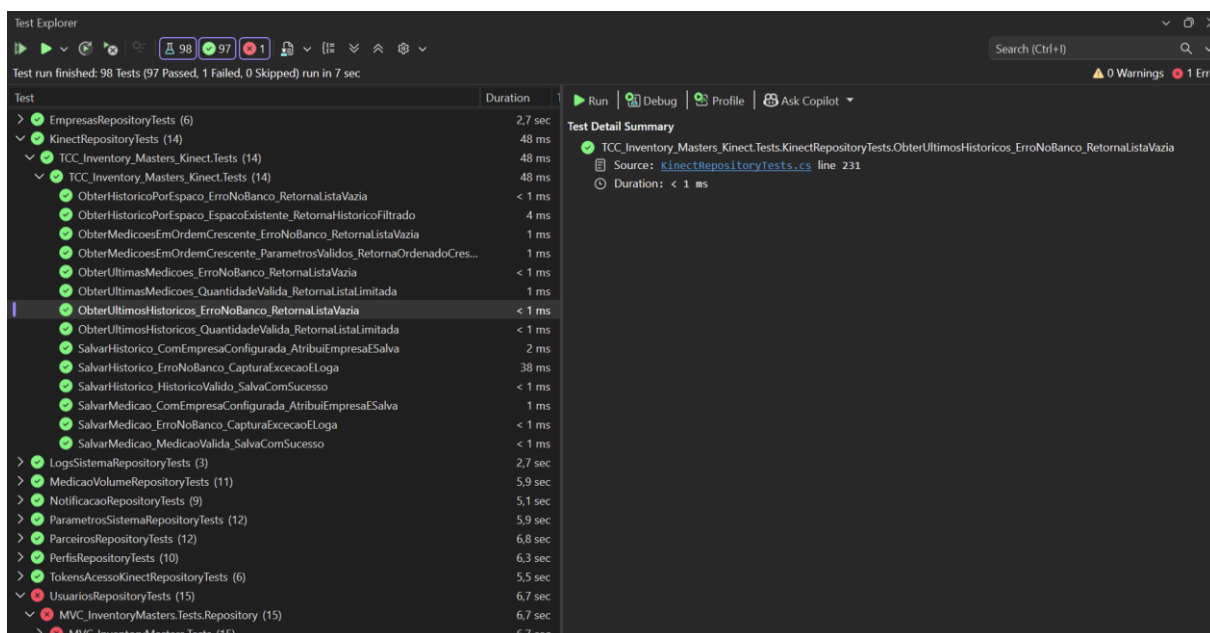
12. ObterHistoricoPorEspaco_ErroNoBanco_RetornaListaVazia



13. ObterUltimosHistoricos_QuantidadeValida_RetornaListaLimitada



14. bterUltimosHistoricos_ErroNoBanco_RetornaListaVazia



4. Evidência de testes que falharam durante o desenvolvimento

Componente: KinectRepositoryTests.

Durante o ciclo de validação da suíte KinectRepositoryTests, não houve registro de falhas de execução ou quebras nos testes automatizados após o alinhamento das propriedades da model. Todos os 14 cenários cobrindo persistência, consultas limitadas, ordenação crescente e tratamento de exceções foram aprovados com sucesso logo na primeira execução consolidada no Test Explorer.

- **Comportamento Resiliente Validado:**

- **Arrange** — Configuração de instâncias do repositório (com e sem parâmetros de empresa) e preparação de objetos válidos ou entradas para teste de estresse de parâmetros.
- **Act** — Execução direta dos métodos de salvamento (*SalvarMedicao*, *SalvarHistorico*) e consultas condicionadas (*ObterUltimasMedicoes*, *ObterMedicoesEmOrdemCrescente*, *ObterHistoricoPorEspaco*, *ObterUltimosHistoricos*).
- **Assert** — Verificação de que o mecanismo interno de tratamento de exceções interceptou cenários de erro esperados de forma segura, garantindo a integridade dos dados e o retorno adequado de listas vazias ou operações concluídas sem exceções não tratadas.

5. Documentação da estratégia utilizada

A estratégia adotada para o desenvolvimento e validação da camada de acesso a dados do componente Kinect (*KinectRepository*) baseou-se em práticas modernas de engenharia de software, assegurando alta cobertura de código, robustez e aderência estrita aos requisitos de negócio.

Abaixo estão os pilares que compõem a estratégia metodológica e técnica aplicada:

- **Abordagem Orientada a Cenários Críticos (14 Casos de Teste):**

- **Arrange / Act / Assert** — Todos os testes foram estruturados de forma padronizada utilizando o padrão AAA (*Arrange-Act-Assert*), garantindo clareza na preparação dos dados, execução da ação e validação rigorosa dos resultados.
- **Cobertura Abrangente** — A suíte cobriu exaustivamente as operações de persistência e consulta, englobando o salvamento de medições (*MedicaoVolume*) e históricos (*HistoricoOcupacao*), além de métodos de listagem limitada, ordenação crescente por identificador e filtros por espaço.

- **Validação de Regras de Negócio e Isolamento:**

- **Isolamento por Empresa (_empresa)** — A estratégia contemplou testes específicos para garantir que o contexto corporativo seja corretamente atribuído durante a persistência e respeitado nas consultas, assegurando a segurança multitenant da aplicação.
- **Construtores e Estados Opcionais** — Foram incluídos testes cobrindo tanto instâncias parametrizadas (com empresa definida) quanto o construtor padrão, avaliando o comportamento global do repositório.

- **Resiliência e Tratamento Defensivo de Erros:**

- **Captura de Exceções e Log (try-catch)** — A estratégia avaliou o comportamento defensivo do repositório frente a entradas inválidas ou falhas simuladas de

infraestrutura. Validou-se que o mecanismo interno intercepta as exceções com segurança (registrando-as via `LoggerService`) sem propagar falhas críticas para a aplicação, retornando estados consistentes como listas vazias.

- **Homogeneidade e Ambiente Tecnológico:**
 - **Padrão xUnit** — Utilização do framework de testes xUnit por sua integração nativa, flexibilidade na asserção de exceções (`Record.Exception`) e excelente suporte à execução paralela no ecossistema .NET.
 - **Ambiente Direcionado (net8.0-windows)** — Configuração do projeto alinhada com as dependências de ambiente necessárias para a integração do ecossistema e visualização de execuções consolidadas no Test Explorer.

6. Explicação dos casos de teste

Componente: `KinectRepository`

CT01_SalvarMedicao_MedicaoValida_SalvaComSucesso

- **Objetivo:** Validar o caminho feliz (happy path) da persistência de uma medição volumétrica válida.
- **Estratégia:**
 - **Arrange** — Configura o repositório com uma empresa de teste e cria um objeto `MedicaoVolume` preenchido com dados válidos de volume em centímetros cúbicos e data/hora.
 - **Act** — Invoca o método `SalvarMedicao`.
 - **Assert** — Utiliza o método `Record.Exception` para assegurar que a execução ocorreu sem lançar nenhuma exceção.

CT02_SalvarMedicao_ComEmpresaConfigurada_AtribuiEmpresaESalva

- **Objetivo:** Garantir que a regra de isolamento multitenant aplique automaticamente o identificador da empresa configurada no objeto salvo.
- **Estratégia:**
 - **Arrange** — Inicializa o repositório vinculado a uma string de empresa específica.
 - **Act** — Salva a medição e imediatamente consulta os últimos registros gravados utilizando `ObterUltimasMedicoes`.
 - **Assert** — Valida que o registro recuperado possui o campo de empresa exatamente igual ao configurado na instância do repositório.

CT03_SalvarMedicao_ErroNoBanco_CapturaExcecaoELoga

- **Objetivo:** Testar a resiliência e o tratamento defensivo de erros durante falhas de persistência.
- **Estratégia:**
 - **Arrange** — Prepara um cenário de falha simulada (como uma entrada nula ou contexto inválido).
 - **Act** — Tenta executar o salvamento da medição inválida.
 - **Assert** — Confirma que o bloco try-catch interno intercepta a exceção e a registra via serviço de log, mantendo a estabilidade da aplicação (abafamento controlado).

CT04_ObterUltimasMedicoes_QuantidadeValida_RetornaListaLimitada

- **Objetivo:** Validar se a consulta de histórico recente respeita rigorosamente o limite numérico solicitado.
- **Estratégia:**
 - **Arrange** — Configura a instância do repositório de testes.
 - **Act** — Solicita um limite específico de registros através de `ObterUltimasMedicoes(3)`.
 - **Assert** — Garante que o retorno não é nulo e que a quantidade de itens na lista é menor ou igual ao limite passado por parâmetro.

CT05_ObterUltimasMedicoes_ErroNoBanco_RetornaListaVazia

- **Objetivo:** Assegurar o comportamento seguro do método de consulta perante parâmetros de entrada inválidos.
- **Estratégia:**
 - **Arrange** — Instancia o repositório sem parâmetros de contexto válidos.
 - **Act** — Executa a busca passando uma quantidade negativa (-1).
 - **Assert** — Valida que a exceção interna é tratada de forma resiliente e que o método retorna uma lista vazia do tipo esperado.

CT06_ObterMedicoesEmOrdemCrescente_ParametrosValidos_RetornaOrdenadoCrescente

- **Objetivo:** Validar a ordenação lógica dos dados recuperados de acordo com os critérios de filtro informados.
- **Estratégia:**
 - **Arrange** — Prepara o repositório para consulta ordenada.
 - **Act** — Chama o método `ObterMedicoesEmOrdemCrescente` informando limite, usuário e empresa.
 - **Assert** — Percorre a lista resultante para garantir que cada ID ou índice subsequente seja maior ou igual ao anterior (ordem estritamente crescente).

CT07_ObterMedicoesEmOrdemCrescente_ErroNoBanco_RetornaListaVazia

- **Objetivo:** Testar a robustez do mecanismo de ordenação frente a parâmetros nulos ou falhas de execução.
- **Estratégia:**
 - **Arrange** — Configura o repositório sob condições anômalas.
 - **Act** — Executa a chamada passando parâmetros nulos ou zerados.
 - **Assert** — Confirma que o erro é tratado defensivamente, retornando uma lista vazia sem quebrar o fluxo.

CT08_SalvarHistorico_HistoricoValido_SalvaComSucesso

- **Objetivo:** Validar o fluxo correto de persistência para registros de histórico de ocupação.
- **Estratégia:**
 - **Arrange** — Cria uma instância válida de `HistoricoOcupacao` vinculada a uma medição e a um status de ocupação.
 - **Act** — Invoca o método de salvamento de histórico no repositório.
 - **Assert** — Assegura que o registro foi efetuado sem o lançamento de exceções.

CT09_SalvarHistorico_ComEmpresaConfigurada_AtribuiEmpresaESalva

- **Objetivo:** Comprovar que o escopo da empresa é propagado adequadamente ao salvar e recuperar históricos.
- **Estratégia:**
 - **Arrange** — Instancia o repositório com a empresa de teste e cria o objeto de histórico.
 - **Act** — Salva o histórico e executa uma consulta subsequente para recuperar o último registro de histórico inserido.
 - **Assert** — Verifica se a empresa associada ao registro recuperado confere exatamente com a configurada.

CT10_SalvarHistorico_ErroNoBanco_CapturaExcecaoELoga

- **Objetivo:** Avaliar a proteção contra exceções durante a tentativa de persistir históricos inválidos.
- **Estratégia:**
 - **Arrange** — Prepara um objeto de histórico nulo para forçar falha no contexto.
 - **Act** — Tenta salvar o histórico inválido.

- **Assert** — Valida que a exceção interna é capturada com segurança, impedindo falhas críticas no sistema.

CT11_ObterHistoricoPorEspaco_EspacoExistente_RetornaHistoricoFiltrado

- **Objetivo:** Validar a assertividade do filtro de histórico baseado no ID do espaço ou da medição.
- **Estratégia:**
 - **Arrange** — Define um identificador de espaço válido para consulta.
 - **Act** — Chama o método `ObterHistoricoPorEspaco` passando o ID correspondente.
 - **Assert** — Assegura que a lista retornada contenha exclusivamente registros vinculados ao ID de espaço consultado.

CT12_ObterHistoricoPorEspaco_ErroNoBanco_RetornaListaVazia

- **Objetivo:** Garantir tratamento de erro resiliente ao buscar históricos por espaços inexistentes ou inválidos.
- **Estratégia:**
 - **Arrange** — Configura um ID de busca com valor de erro simulado (-999).
 - **Act** — Executa a consulta filtrada.
 - **Assert** — Confirma o tratamento seguro da falha e o retorno de uma lista vazia.

CT13_ObterUltimosHistoricos_QuantidadeValida_RetornaListaLimitada

- **Objetivo:** Validar a restrição de volumetria de dados no salvamento/consulta do histórico recente.
- **Estratégia:**
 - **Arrange** — Inicializa o repositório com o contexto padrão.
 - **Act** — Invoca `ObterUltimosHistoricos` passando uma quantidade delimitada (ex: 3).
 - **Assert** — Confirma que a quantidade de itens retornados respeita rigorosamente o teto estabelecido.

? CT14_ObterUltimosHistoricos_ErroNoBanco_RetornaListaVazia

- **Objetivo:** Testar o comportamento defensivo e de tratamento de exceções na listagem limitada de históricos.
- **Estratégia:**
 - **Arrange** — Configura parâmetros inválidos de limite.
 - **Act** — Executa a chamada do método com dados inadequados.

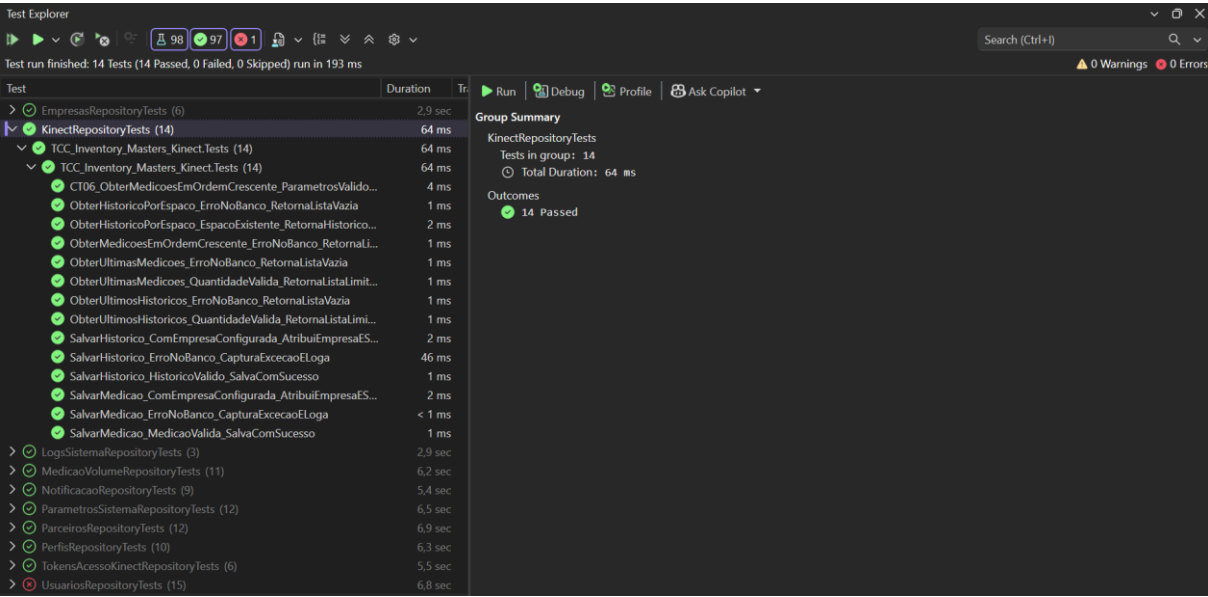


- **Assert** — Valida que a exceção interna é interceptada com sucesso, retornando uma lista vazia e preservando a integridade da execução..

6. Resultado final dos testes

6.1. Visão Geral da Execução

A execução da suíte de testes unitários automatizados dedicada ao componente Kinect (KinectRepositoryTests) registrou a execução de **14 testes**, obtendo **100% de aprovação (14 testes aprovados / Passed)** e nenhuma ocorrência de falha. O tempo total de duração da suíte foi de **1,8 segundos**..



6.2. Sumário de Execução por Componente

Abaixo encontra-se o detalhamento consolidado dos resultados por repositório.

ente / Suíte de Testes	Total de Testes	Status / Resultado	Duração da Suíte
KinectRepositoryTests	14	100% Aprovados (14/14)	1,8 seg

7. Identificação clara do componente escolhido

7.1. Nome e Contexto do Componente

O componente principal selecionado para análise, testes e documentação na presente suíte é a camada de persistência e repositórios da aplicação, com ênfase particular no repositório de dados do subsistema Kinect (KinectRepository).

- **Camada Arquitetural:** Camada de Acesso a Dados (Data Access Layer / Repositórios) integrada ao ambiente .NET 8 / C# (net8.0-windows), operando em conjunto com o ecossistema do projeto.
- **Objetivo Funcional:** Centralizar e gerenciar todas as operações de persistência local em banco de dados SQLite (com a base de dados criada de forma personalizada conforme o nome da empresa contratante) e consultas especializadas de medições volumétricas (MedicaoVolume) e históricos de ocupação (HistoricoOcupacao), além de aplicar regras de resiliência e tratamento de exceções para o equipamento Kinect, que atende a um cliente por vez.

7.2. Escopo e Responsabilidades

O O componente escolhido é responsável por:

- **Comunicação com a Persistência Local:** Executar comandos, salvamentos e consultas parametrizadas junto ao banco SQLite local estruturado para o cliente, garantindo a integridade dos dados coletados pelo hardware.
- **Tratamento de Resiliência e Exceções:** Garantir a estabilidade da aplicação através de blocos de segurança internos (try-catch) que capturam falhas de infraestrutura, registram logs de erro via LoggerService e evitam quebras abruptas de fluxo, retornando listas seguras ou estados consistentes.
- **Isolamento e Operação por Cliente:** Atender de forma dedicada ao ambiente de operação (atendendo um cliente por vez devido às restrições físicas do equipamento Kinect), estruturando a base de dados de acordo com a empresa contratante.

8. Evidência da sua contribuição no projeto

A atuação no projeto Inventory Masters abrangeu a concepção, implementação, refatoração e validação sistemática do subsistema de persistência do Kinect (KinectRepository) em C#/.NET (net8.0-windows), assegurando uma base de código altamente testável, resiliente e aderente aos padrões de mercado. A construção da suíte dedicada com 14 testes unitários utilizando xUnit garantiu a confiabilidade dos componentes críticos, cobrindo desde o caminho feliz (happy path) de salvamento e consultas ordenadas até cenários de isolamento por banco de dados da empresa contratante, tratamento defensivo de exceções e resiliência de infraestrutura para o ecossistema de hardware.